

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”

Кафедра «Интеллектуальные информационные технологии»

ЛАБОРАТОРНАЯ РАБОТА №1

по дисциплине «Аппаратное и программное обеспечение сетей» за 4 семестр

Тема: «Организация ТСП – сервера и ТСП – клиента»

Вариант 5

Выполнил:
студент 3 курса
группы ИИ-28
(1 подгруппа)
Деркач Е. В.
Проверил:
Кулик А.Д.

Цель работы: изучить Winsock2 и написать TCP-сервер и TCP-клиент, которые обрабатывают поток символов группами по 64 и рвут связь, если в группе меньше 3 разных символов.

Ход работы

Постановка задачи:

Задания для реализации TCP (UDP) сервера: после приема каждой группы из 64 символов от клиента сервер формирует и отправляет клиенту цепочку <«символ1-количество1», «символ2-количество2» и т.д.>, где «символ» - встреченный в группе символ, а «количество» - их количество в группе. Если в очередной группе число разных символов меньше 3, то отправляется соответствующее сообщение клиенту и сервер разрывает соединение.

Задания для реализации TCP (UDP) клиента:

Ввод символов с отсылкой введенной строки по нажатию на клавишу	Ведение файла протокола событий, включающих: 1) время начала и окончания соединения; 2) передаваемую серверу строку и время передачи строки; 3) принимаемую от сервера строку и время приема строки.	Возможность разрыва соединения при помощи команды: disconnect <адрес> <порт>	1) Задание в программе клиента специальной команды и параметров: подключения к серверу: connect <адрес> <порт> 2) Автоматическое подключение к серверу с заданным по умолчанию адресом при запуске клиента
PgDn	1), 2), 3)	-	2)

Код программы:

server.cpp

```
#include <stdio.h>
#include <string.h>
#include <winsock2.h>
#include <windows.h>

#define MY_PORT 666

#define PRINTNUSERS if (nclients)\
    printf("%d user on-line\n",nclients);\
    else printf("No User on line\n");

DWORD WINAPI WorkWithClient(LPVOID client_socket);

int nclients = 0;

int main(int argc, char* argv[])
{
```

```

char buff[1024];

printf("SERVER\n\n");

if (WSAStartup(0x0202, (WSADATA *) &buff[0]))
{
    printf("Error WSAStartup %d\n", WSAGetLastError());
    return -1;
}

SOCKET mysocket;
if ((mysocket=socket(AF_INET, SOCK_STREAM, 0)) == INVALID_SOCKET)
{
    printf("Error socket %d\n", WSAGetLastError());
    WSACleanup();
    return -1;
}

sockaddr_in local_addr;
local_addr.sin_family=AF_INET;
local_addr.sin_port=htons(MY_PORT);
local_addr.sin_addr.s_addr=0;

if (bind(mysocket, (sockaddr *) &local_addr, sizeof(local_addr)))
{
    printf("Error bind %d\n", WSAGetLastError());
    closesocket(mysocket);
    WSACleanup();
    return -1;
}

if (listen(mysocket, 0x100))
{
    printf("Error listen %d\n", WSAGetLastError());
    closesocket(mysocket);
    WSACleanup();
    return -1;
}

printf("Waiting for connections, port %d\n", MY_PORT);

SOCKET client_socket;
sockaddr_in client_addr;
int client_addr_size=sizeof(client_addr);

while((client_socket=accept(mysocket, (sockaddr *)
    &client_addr, &client_addr_size)) != INVALID_SOCKET)
{
    nclients++;

    HOSTENT *hst;
    hst=gethostbyaddr((char *)&client_addr.sin_addr.s_addr, 4, AF_INET);

    printf("+%s [%s] new connect!\n",
        (hst)?hst->h_name:"",
        inet_ntoa(client_addr.sin_addr));
}

```

```

        PRINTNUSERS

        DWORD thID;
        CreateThread(NULL, 0, WorkWithClient,
                     (LPVOID)client_socket, 0, &thID);
    }
    return 0;
}

DWORD WINAPI WorkWithClient(LPVOID client_socket)
{
    SOCKET my_sock;
    my_sock=(SOCKET)client_socket;
    char buff[20*1024];
    #define SHELLLO "Hello, Student!\r\n"

    send(my_sock,SHELLLO,strlen(SHELLLO),0);

    char group[64];
    int filled = 0;
    int bytes_recv;

    while( (bytes_recv=recv(my_sock,&buff[0],sizeof(buff),0))
           && bytes_recv !=SOCKET_ERROR)
    {
        int i;
        for (i = 0; i < bytes_recv; i++)
        {
            group[filled++] = buff[i];
            if (filled < 64)
                continue;

            int counts[256];
            int order[256];
            int nunique = 0;
            int k;
            memset(counts, 0, sizeof(counts));
            for (k = 0; k < 64; k++)
            {
                unsigned char ch = (unsigned char)group[k];
                if (counts[ch] == 0)
                    order[nunique++] = ch;
                counts[ch]++;
            }

            char outmsg[1024];
            if (nunique < 3)
            {
                strcpy(outmsg,
                      "Less than 3 distinct characters. Connection
closed.\r\n");
                send(my_sock, outmsg, strlen(outmsg), 0);
                nclients--;
                printf("-disconnect (less than 3 distinct characters)\n");
                PRINTNUSERS
                closesocket(my_sock);
            }
        }
    }
}

```

```

        return 0;
    }

    strcpy(outmsg, "<");
    for (k = 0; k < nunique; k++)
    {
        char piece[64];
        unsigned char ch = (unsigned char)order[k];
        char shown[8];
        if (ch == '\n') strcpy(shown, "\\n");
        else if (ch == '\r') strcpy(shown, "\\r");
        else if (ch == '\t') strcpy(shown, "\\t");
        else { shown[0] = (char)ch; shown[1] = 0; }
        sprintf(piece, "%s'%s-%d'",
            k ? ", " : "",
            shown, counts[ch]);
        strcat(outmsg, piece);
    }
    strcat(outmsg, ">\r\n");
    send(my_sock, outmsg, strlen(outmsg), 0);
    printf("group processed: %s", outmsg);
    fflush(stdout);
    filled = 0;
}

nclients--;
printf("-disconnect\n");
PRINTNUSERS
closesocket(my_sock);
return 0;
}

```

client.cpp

```

#include <stdio.h>
#include <string.h>
#include <time.h>
#include <conio.h>
#include <winsock2.h>
#include <windows.h>

#define PORT 666
#define SERVERADDR "127.0.0.1"

SOCKET g_sock = INVALID_SOCKET;
FILE *g_log = NULL;
int g_connected = 0;

void log_event(const char *what, const char *detail)
{
    time_t t;
    struct tm *tm;
    char ts[64];
    if (!g_log)
        return;

```

```

time(&t);
tm = localtime(&t);
sprintf(ts, "%04d-%02d-%02d %02d:%02d:%02d",
        tm->tm_year + 1900, tm->tm_mon + 1, tm->tm_mday,
        tm->tm_hour, tm->tm_min, tm->tm_sec);
fprintf(g_log, "%s %s %s\n", ts, what, detail ? detail : "");
fflush(g_log);
}

DWORD WINAPI RecvThread(LPVOID)
{
    char buff[4096];
    int n;
    while ((n = recv(g_sock, &buff[0], sizeof(buff) - 1, 0)) > 0)
    {
        buff[n] = 0;
        printf("S=>C:%s", buff);
        if (n > 0 && buff[n - 1] != '\n')
            printf("\n");
        log_event("RECEIVED", buff);
    }
    printf("\nConnection closed by server\n");
    log_event("CONNECTION END", "");
    g_connected = 0;
    return 0;
}

int main(int argc, char *argv[])
{
    char buff[1024];

    printf("CLIENT\n\n");
    printf("Auto-connect to %s:%d\n", SERVERADDR, PORT);
    printf("Type text, send - PgDn or Enter, exit - quit/exit + PgDn or Enter\n\n");

    g_log = fopen("tcp-client.log", "a");

    if (WSAStartup(0x202, (WSADATA *)&buff[0]))
    {
        printf("WSAStart error %d\n", WSAGetLastError());
        return -1;
    }

    g_sock = socket(AF_INET, SOCK_STREAM, 0);
    if (g_sock == INVALID_SOCKET)
    {
        printf("Socket() error %d\n", WSAGetLastError());
        return -1;
    }

    sockaddr_in dest_addr;
    dest_addr.sin_family = AF_INET;
    dest_addr.sin_port = htons(PORT);
    HOSTENT *hst;

```

```

if (inet_addr(SERVERADDR) != INADDR_NONE)
    dest_addr.sin_addr.s_addr = inet_addr(SERVERADDR);
else if (hst = gethostbyname(SERVERADDR))
    ((unsigned long *)&dest_addr.sin_addr)[0] =
        ((unsigned long **)hst->h_addr_list)[0][0];
else
{
    printf("Invalid address %s\n", SERVERADDR);
    closesocket(g_sock);
    WSACleanup();
    return -1;
}

if (connect(g_sock, (sockaddr *)&dest_addr, sizeof(dest_addr)))
{
    printf("Connect error %d\n", WSAGetLastError());
    return -1;
}

printf("Connection to %s established successfully\n", SERVERADDR);
log_event("CONNECTION START", SERVERADDR ":666");
g_connected = 1;

DWORD thID;
CreateThread(NULL, 0, RecvThread, NULL, 0, &thID);

char line[1024];
int len = 0;
printf("S<=C:");
fflush(stdout);

while (g_connected)
{
    int c = getch();
    if (c == 27)
        break;
    if (c == 0 || c == 224)
    {
        int k = getch();
        if (k == 81 && len > 0)
        {
            line[len] = 0;
            if (!strcmp(line, "quit") || !strcmp(line, "exit"))
                break;
            send(g_sock, line, len, 0);
            printf("\nsent %d chars.\nS<=C:", len);
            log_event("SENT", line);
            len = 0;
        }
        continue;
    }

    if (c == '\r' || c == '\n')
    {
        if (len > 0)
        {

```

```

        line[len] = 0;
        if (!strcmp(line, "quit") || !strcmp(line, "exit"))
            break;
        send(g_sock, line, len, 0);
        printf("\nsent %d chars.\nS<=C:", len);
        log_event("SENT", line);
        len = 0;
    }
    continue;
}

if (c == 8)
{
    if (len > 0)
    {
        len--;
        printf("\b \b");
        fflush(stdout);
    }
    continue;
}
if (c >= 32 && len < (int)sizeof(line) - 1)
{
    line[len++] = (char)c;
    putchar(c);
    fflush(stdout);
}
}

log_event("CONNECTION END", "");
closesocket(g_sock);
WSACleanup();
if (g_log)
    fclose(g_log);
return 0;
}

```

Тестирование программы:

```
Waiting for connections, port 666
+genuine-software.autodesk.com [127.0.0.1] new connect!
1 user on-line
group processed: <'a-19', 'b-23', 'c-9', 'v-3', 'd-2', 'f-1', 'j-1', 'h-1', 'l-1', 't-2', 'i-1', 'r-1'>
-disconnect
No User on line
```

```
PS D:\Course 3\ai\pos> g++ client.cpp -o client.exe -lws2_32
PS D:\Course 3\ai\pos> .\client
CLIENT
```

```
Connection to 127.0.0.1 established successfully
S<=C:S=>C:Hello, Student!
aaaaaaaaaaaaabbbbbbbbbbbbbbbbbbbccccccccc
sent 45 chars.
S<=C:aabavbvadfjhltidrt
sent 19 chars.
S<=C:S>C:<'a-19', 'b-23', 'c-9', 'v-3', 'd-2', 'f-1', 'j-1', 'h-1', 'l-1', 't-2', 'i-1', 'r-1'>
aaaaaaaaaaaaaassssssssssssdddddffffffffff
sent 55 chars.
S<=C:exit
Connection closed by server
PS D:\Course 3\aiapos>
```

```
tcp-client.log
1 2026-09-22 12:04:59 CONNECTION START 127.0.0.1:666
2 2026-09-22 12:04:59 RECEIVED Hello, Student!
3
4
5 2026-09-22 12:05:05 SENT aaaaaaaaaaaaaabbbbbbbbbbbbbbbbbbccccccccc
6 2026-09-22 12:05:09 SENT aabavbvvd fjhl tidrt
7 2026-09-22 12:05:09 RECEIVED <'a-19', 'b-23', 'c-9', 'v-3', 'd-2', 'f-1', 'j-1', 'h-1', 'l-1',
't-2', 'i-1', 'r-1'>
8
9
10 2026-09-22 12:05:17 SENT aaaaaaaaaaaaaasssssssssssssdddddffffffffff
11 2026-09-22 12:05:21 CONNECTION END
12 2026-09-22 12:05:21 CONNECTION END
13
```

Вывод: изучил функции Winsock2 (WSAStartup, socket, bind, listen, accept, connect, send, recv, closesocket, WSACleanup); написал и отладил сервер и клиент для варианта 5; сервер считает вхождения символов в каждой группе из 64 и разрывает соединение при <3 разных символах, клиент автоматически подключается к 127.0.0.1:666, отправляет строки по PgDn/Enter и ведёт лог tcp-client.log; программы проверены, задание выполнено полностью.